Rapport prosjekt I ressurseffektive programmer

Oppgave 1:
Først valgte jeg «ID» som primary key hos lego_set. Hos lego_brick lagde jeg en composite key (brick_type_id, color_id). Til slutt ga jeg inventory en composite key (set_id, brick_type_id, color_id). Rekkefølgen valgte jeg slik at den kutter ut så mye volum som mulig, så tidligist som mulig. Ved å lete etter set først blir det mye raskere å finne en spesifik brikke i settet. Da bruker vi ikke prosseserinskraft på å lete gjennom overflødig informasjon, som gjør prossesen raskere å mer effektiv.

Oppgave 2:
Jeg lagde to indekser som dekker situasjonene i oppgaven.

```
lego-db=# CREATE INDEX idx_inventory_type_set ON lego_inventory (brick_type_id, set_id);
CREATE INDEX
```

```
lego-db=# CREATE INDEX idx_inventory_color_set ON lego_inventory (color_id, set_id);
CREATE INDEX
```

Og fikk en hastighet som er mer enn 30x raskere fra 690.34ms til 19.52ms. Grunnen til denne forbedringen er et nå har vi også muligheten til å søke etter brick_type og color_id først. Dette gjør at vi essensielt har to ekstra søkefelt som er sortert etter brikker så maskinen f.eks. ikke lenger trenger å gå gjennom hvert set individuelt for å finne ut om det inneholder x brikke. Nå tar den i stedet å sorterer etter alle brikke x og tar alle set som er linket til brikken.
Før:

```
lego-db=# SELECT DISTINCT set_id FROM lego_inventory WHERE brick_type_id = '3001';
Time: 690.387 ms
```

Etter:

```
lego-db=# SELECT DISTINCT set_id FROM lego_inventory WHERE brick_type_id = '3001';
Time: 19.520 ms
```

Oppgave 3: Den originale koden har kompleksiteten O(n^2). Dette var fordi hver gang et nytt set ble lagt til måtte python lage en helt ny string ved å kopiere alle de tidligere settene og legge til den nye på slutten. Dette skjer fordi en string i python ikke kan redigeres etter at den har blitt lagd og må derfor lage en helt ny en for å bytte ut den gamle. Jeg fikset dette problemet ved å legge alle radene til i en liste, dette gjør at jeg kan legge til nye set i listen med .join funksjonen uten å måtte skrive om samme string igjen og igjen. Den nye kompleksiteten blir da O(n).

Oppgave 4: Jeg starter med å importere gzip og fikser file handle leaks ved å bytte alle instanser av «open()» til «with open». Dette sørger basically bare for at python lukker filer den har åpnet når den er ferdig med dem.
Jeg bruker så request.args til å sørge for at encoding parameteren holder seg til utf-8 og eller utf-16 uansett hva. Andre verdier defaulter til utf-8 og .lower sørger for at alt er lowercase for QOL.

For å sørge for at det ikke blir noen kræsj med meta charset og utf-16 fjerner jeg charset meta når encodingen ikke er utf-8. Dette velger jeg for å holde encoding konsekvent fra nettleser til nettleser.

Til slutt bruker jeg gzip for å komprimere html siden som har blitt encoded som en byte streng. Dette gjør filen mye mindre og lettere å overføre over nett. Til slutt returnerer jeg en respons og sørger for at jeg minner nettleseren på å dekomprimere body før den prøver å komprimert ikke-lesbart søppel.

Oppgave 5:
I JSON delen av oppgaven har jeg først en check som fanger alle queries som går ikke passer innenfor parameteren, altså alle uidentifiserte Ider. Deretter fetcher jeg set informasjonen fra databasen og setter dem i en rekkefølge jeg syntes var mest logisk som kunde. Så fetcher jeg inventory dataen og joiner lego_brikke og lego_inventory sammen slik at jeg enklere kan ta informasjonen fra de begge og til slutt setter jeg opp et nestet json struktur og returnerer resultatet.

I Binary fil delen starter jeg med å lage en 8 byte header. 4 bytes går til å skrive LEGO og 1 går til hvilken versjon det er. Dette fungerer funksjonelt som en nøkkel som lar maskinen vite at det er riktig fil som kommer inn. De siste 3 bytesa benyttes ikke, de spares i tilfelle programmet utvides senere. Deretter enkoder jeg navnet, og jeg sørger for at lengden på pakken blir notert. Dette er viktig for dekoding senere. Så tar jeg inn all den andre informasjonen og gjør de om til bytes på samme måte som jeg gjorde tidligere (år, kategori, etc). Deretter lager jeg en liknende formel for å enkode de individuelle brikkene i inventory databasen og jeg teller hvor mange brikker som som er i inventoryen også. Deretter lager jeg en løkke for antall brikker i inventory og pakker de ned en etter en også. Dermed lager jeg en reader-fil som gjør mer eller mindre alle stegene i revers. Den tar inn den krypterte dataen og lengden/antallet/størrelsen på binærnøkkelen og dekrypterer dataen ved å lese dataen med konteksten vi lagde når vi pakket den inn. Denne dataen blir så printet ut i terminalen på en lesbar måte for brukeren.

Oppgave 6: Cachet bruker OrderedDict for å lagre lego set som har blitt brukt nylig. Cache lagrer dataen i minnet slik at forespørsler etter de samme settene ikke trenger å spørre databasen på nytt det er plass til 100 sett i cachet. Jeg bruker Least Recently Used (LRU) som eviction policy, som betyr at når cachet er fullt er det det settet som ble brukt senest bak i tid. Alle operasjonene til dette cachet fungerer i konstant tid som betyr at kompleksiteten holder seg på o(1)

Når endpoint ikke er cachet tok det 65.48 millisekunder.

```
127.0.0.1 - - [27/Mar/2026 20:52:46] "GET /set?id=10001-1 HTTP/1.1" 200 -
CACHE MISS for 10001-1: 65.48ms (Database query)
```

Når endpoint var cachet tok det 0.07 millisekunder.

```
CACHE HIT for 10001-1: 0.07ms
127.0.0.1 - - [27/Mar/2026 20:53:16] "GET /api/set?id=10001-1 HTTP/1.1" 200 -
```

Oppgave 7:
Jeg lager en database wrapper klasse som inneholder alle psycopg og andre funksjoner som har interaksjoner med legoDB databasen min. Så går jeg gjennom alle end pointene mine og restrukturerer dem med den nye database klassen for å gjøre dem at de ser mindre klunkete ut. Endpoints håndterer nå kun requests og responses og psycopg har ingen direkte bruk i

funksjonene. Jeg lagde også en test fil som kjører en liten test på alle de forskjellige endpointene